1. Introduction

This work implements an occupancy grid mapping algorithm coded in C++. The robot's sensor is made up of four ultrasonic sensors and odometry which is capable to make a 2D map of the environment
from the measurements the robot collects.

The program:

• Reads in robot's position, orientation, and ultrasonic measurements from a CSV file.

• Updates a probabilistic occupancy grid using a log-odds representation.

• Exports the map as an image (PGM) as well as a CSV table of probabilities.

• A Python script animates how the map progresses in C++.

2. Methodology

2.1 Input Handling

The program reads robot.csv that contain columns for position (pos_x, pos_y), heading, and four ultrasonic measurements.

- The ultrasonic values are time of flight readings.
- They are converted to distances using the speed of sound (343 m/s) and divided by two to account for round trip.

Each row is stored as:

{x, y, heading, dist0, dist1, dist2, dist3}

2.2 Grid Setup

- Resolution: 0.05 m per cell (5 cm).
- Padding: 1 m added around the robot's explored area.
- Map storage: 2D grid of log-odds values initialized to 0.

This ensures a good tradeoff between map detail and memory efficiency.

2.3 Sensor Placement and Orientation

Each ultrasonic sensor is modeled by its position and orientation relative to the robot. Example: the front-right sensor is at (0.10, 0.05) with angle +45°.

The global sensor positions are updated using the robot’s heading with a standard 2D rotation transform:

2.4 Cone Model with Multiple Rays

Ultrasonic sensors range a cone of 30°. In order to account for this the program simulates 9 rays per cone. Each ray is equally spaced with ±15° around the beam center.

- Too few rays means you may miss some obstacle.
- Too many rays make it inefficient.
- Nine rays were chosen as a compromise and that gave me the ideal map.

2.5 Ray Tracing

Each ray is quantized into grid cells (as per Bresenham's line algorithm).

- Cells along ray path are updated as free
- Final cell (if within range) is updated as occupied.

There is consideration for both free space and obstacles.

2.6 Log-Odds Probability Update

Cell updates are based on:

- Occupied probability:
- Free probability:

Converted into log-odds:

,

- : strong confidence when an obstacle is detected.
- : weaker confidence in free space (absence of detection is less reliable).

At the end, log-odds are converted back to probabilities for visualization:

3. Results

3.1 Occupancy Grid

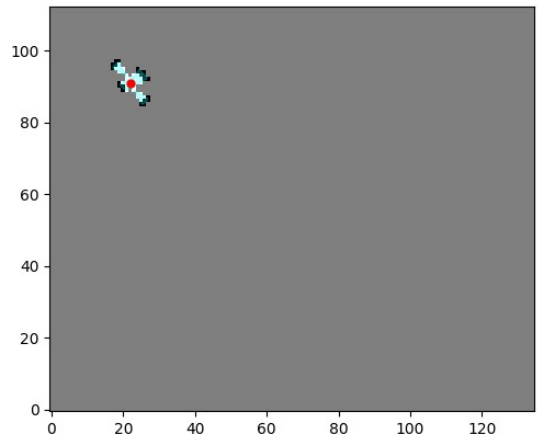
The program outputs an occupancy grid as a grayscale image:

- Dark = occupied.
- Light = free.



3.2 Animation

A Python script was created to animate the mapping process, illustrating how the occupancy grid develops as measurements are provided. I had put in this part to illustrate how the algorithm considers every step.



4. Conclusion

The project demonstrates how ultrasonic sensor data can be integrated into a probabilistic occupancy grid using log-odds.

Key design choices:

- Modeling each ultrasonic cone with 9 rays.

- Using log-odds update with asymmetric parameters ()
- Outputting both visual (PGM) and numeric (CSV) maps.

5. References

- Thrun, S., Burgard, W., & Fox, D. Probabilistic Robotics. MIT Press, 2005.
- Siegwart, R., Nourbakhsh, I. R., & Scaramuzza, D. (2011). Introduction to Autonomous Mobile Robots (2nd ed.). MIT Press.
- Red Blob Games – Bresenham's Line Algorithm